

# A Shape-Adaptive Architecture with Disaggregated Quantization for Efficient LLM Serving

Cong Guo\*  
Duke University  
Durham, NC, USA  
cong.guo@duke.edu

Chiyue Wei\*  
Duke University  
Durham, NC, USA  
chiyue.wei@duke.edu

Bowen Duan  
Duke University  
Durham, NC, USA  
bowen.duan@duke.edu

Haoxuan Shan  
Duke University  
Durham, NC, USA  
haoxuan.shan@duke.edu

Benjamin F. Morris III  
Duke University  
Durham, NC, USA  
ben.morris@duke.edu

Yintao He  
Duke University  
Durham, NC, USA  
yintao.he@duke.edu

Hai “Helen” Li  
Duke University  
Durham, NC, USA  
hai.li@duke.edu

Yiran Chen  
Duke University  
Durham, NC, USA  
yiran.chen@duke.edu

## Abstract

Large language models (LLMs) have become the backbone of modern AI applications, but pose significant challenges for efficient inference. Their autoregressive generation divides execution into two phases: *prefill*, dominated by large GEMMs, and *decoding*, dominated by small GEMVs. Modern serving systems further introduce complexity through continuous batching and prefill–decoding disaggregation, leading to dynamic workloads and phase separation. However, existing accelerators remain poorly aligned with these system-level behaviors, resulting in inefficiencies in LLM serving.

In this work, we present **DynaCore**, a unified architecture for efficient LLM serving via system–architecture co-design. We observe that the compute tile a systolic array executes, its *Minimum Efficient Unit* (MEU), spans all three GEMM dimensions. **DynaCore** reshapes the MEU along all three: spatially it trades array width against height *asymmetrically*, raising weight delivery while leaving the input path untouched, and temporally *Split-K* maps the reduction onto the array, folding partial sums through the interconnect the array already has. To exploit phase separation, we further propose *disaggregated quantization*, applying dual-side quantization to prefill and weight-only quantization to decoding, with an inner-product mixed-precision datapath that keeps output width invariant to precision. A *runtime scheduling framework* then selects an MEU per batch. Evaluation with real-world serving traces shows that **DynaCore** substantially reduces service-level latency over quantization and reconfigurable accelerators, improving *TTFT* by 3.50× and 2.97× and *TPOT* by 36.55× and 8.02×, respectively.

## 1 Introduction

Large Language Models (LLMs) [7, 16, 17, 27, 45, 50, 53, 54] have become the foundation of a wide range of real-world applications, including natural language understanding [15, 38, 54, 63] and code generation [52, 58, 64]. As model sizes continue to grow, LLM serving increasingly stresses computing infrastructure, with inference emerging as the primary performance and cost bottleneck. A key

challenge in LLM inference stems from its autoregressive generation paradigm [30, 55, 56], which divides execution into two fundamentally different phases: *prefill* and *decoding* [42]. The prefill phase processes all input tokens in parallel using compute-intensive matrix–matrix multiplications (GEMMs), achieving high utilization on modern accelerators. In contrast, the decoding phase generates tokens sequentially, where each step is dominated by matrix–vector multiplications (GEMVs) with limited parallelism and low arithmetic intensity. This fundamental shift from throughput-oriented to latency-sensitive computation leads to severe hardware under-utilization during decoding, making it the dominant bottleneck in end-to-end LLM serving [2, 34, 41, 42].

To mitigate the inefficiency of the decoding phase, prior works have explored optimizations across both algorithmic and system levels. From an algorithmic perspective, decoding is widely recognized as a memory-bound workload in which weight accesses dominate execution (often accounting for over 90% of memory traffic), while activation footprints remain relatively small [24, 42]. This observation has motivated weight-only quantization techniques [31, 36, 48], which reduce memory bandwidth demand by compressing model weights, thereby improving performance in memory-bound regimes. From a system perspective, continuous batching (CB) [24, 59] improves effective parallelism by aggregating decoding requests. Additionally, when prefill and decoding are co-located on the same device, resource contention leads to significant latency variability. Prefill–decoding disaggregation (PDD) [41, 43, 62] alleviates this interference by separating the two phases across devices, reducing decoding latency.

Despite these advances, existing accelerator designs remain poorly aligned with the combined effects of algorithmic and system-level optimizations in LLM serving. This misalignment arises because prefill and decoding exhibit fundamentally different execution characteristics, yet current designs lack cross-layer co-design across algorithms, systems, and hardware.

First, decoding remains a fundamental bottleneck due to the low hardware utilization of GEMV operations. Modern accelerators, particularly systolic-array-based designs [10, 28, 29, 32, 47], are

\*Both authors contributed equally to this research.

optimized for large GEMMs and exhibit poor efficiency when processing narrow GEMVs. While continuous batching (CB) partially increases parallelism, it introduces highly dynamic GEMM shapes that depend on the number of active tokens, leading to fluctuating execution patterns. As a result, CB does not resolve the architectural mismatch between decoding workloads and hardware execution models (Section 2.3). Achieving high efficiency, therefore, requires architectures that better support fine-grained parallelism and align with decoding-specific computation patterns.

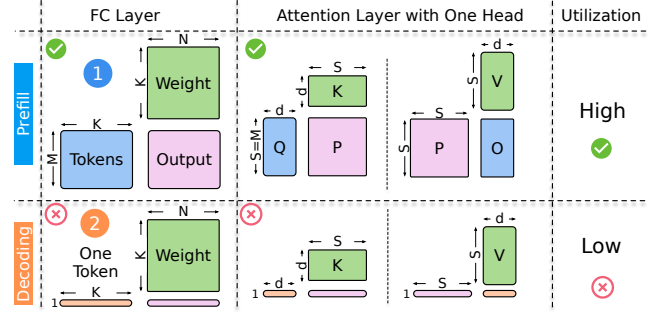
Second, quantization [4, 16, 18, 31, 36, 37, 48, 57], a key technique for improving LLM efficiency, is usually applied with an identical configuration (e.g., W8A8 or W4A16, where  $WmAn$  denotes  $m$ -bit weights and  $n$ -bit activations) across both prefill and decoding phases. However, these two stages exhibit fundamentally different bottlenecks, making a unified design inherently suboptimal (Section 2.4). Dual-side quantization [37, 57] (W8A8) improves compute efficiency and maintains high accuracy during prefill but offers limited benefit for decoding, where performance is constrained by memory bandwidth. In contrast, weight-only quantization [31, 36] (W4A16) reduces memory traffic during decoding but provides little acceleration for prefill and degrades accuracy. This mismatch highlights the need for phase-specific quantization that adapts to the distinct compute and memory characteristics of each stage.

Finally, these challenges highlight the need for a unified system-architecture co-design. Deploying separate hardware for prefill and decoding leads to resource imbalance under dynamic PDD scheduling, while static architectures cannot adapt to CB-induced variability. An effective solution must therefore support both phases within a unified architecture that can dynamically reconfigure its execution behavior and coordinate with system-level scheduling.

In this work, we present *DynaCore*, a phase-adaptive architecture for efficient LLM serving. Our starting point is that the compute tile a systolic array executes, which we call the *Minimum Efficient Unit* (MEU), spans all three dimensions of a GEMM:  $M$  and  $N$  are partitioned *spatially* across the array as the extents  $m$  and  $n$ , while the reduction dimension  $K$  is partitioned *temporally* as the extent  $k$ .

First, we design a *splittable systolic array* that reshapes the array spatially, trading  $m$  against  $n$  at a fixed PE count. The reshaping is *asymmetric*: flattening demands more weight bandwidth while leaving the input path unchanged, matching a workload that has little activation data to move but must stream weights to every PE. Second, we introduce *Split-K*, which partitions the reduction across the array once both spatial extents are pinned by the workload. Since slices of  $K$  must be summed rather than drained independently, we fold them with a *tick-tock* schedule that alternates forwarding and accumulating along the links the array already has, adding no reduction network.

Third, we propose *disaggregated quantization*, which supplies what the first two dimensions demand. Prefill adopts dual-side quantization (W8A8) while decoding adopts weight-only quantization (W4A16), relieving the weight path that asymmetric reshaping stresses; its mixed-precision datapath directs low-precision throughput along  $K$  rather than into additional outputs, leaving every array configuration reachable at either precision. Finally, we develop a *runtime scheduling framework* that selects an MEU



**Figure 1: GEMM characteristics across prefill and decoding phases in LLM.**

for each batch, searching the reachable configurations against a joint latency–energy objective from offline profiling.

*DynaCore* integrates these techniques into a unified design for practical LLM serving under dynamic workloads. Comprehensive evaluation demonstrates significant gains in throughput, energy efficiency, and latency metrics (*time-to-first-token* (TTFT) and *time-per-output-token* (TPOT)) over state-of-the-art LLM accelerators. In summary, our main contributions are as follows:

- We design a *splittable systolic array* that reshapes the MEU along all three dimensions, two spatially and one temporally, without altering the array topology.
- We propose *disaggregated quantization* with an inner-product mixed-precision datapath, supplying the weight bandwidth that reshaping demands.
- We develop a *runtime scheduling framework* that selects an MEU per batch, bridging system-level dynamics and architectural adaptation under continuous batching and disaggregated serving.
- Comprehensive evaluation shows that *DynaCore* substantially reduces service-level latency over the FIGLUT [39] quantization accelerator and the Planaria [19] reconfigurable accelerator, improving TTFT by  $3.50\times$  and TPOT by  $36.55\times$  and  $8.02\times$ , respectively, under real serving traces.

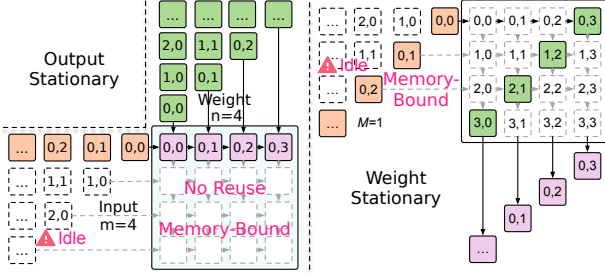
## 2 Background and Motivation

### 2.1 LLM Inference and Characteristics

Large Language Models (LLMs) [7, 17, 27], such as GPT [1, 16, 44, 45] and LLaMA [20, 53, 54], are built on the Transformer architecture [55], comprising stacked attention and feed-forward layers. During inference, LLMs generate text in an *autoregressive* manner.

**Prefill.** Given a prompt, the model first performs a prefill phase, processing all input tokens in parallel across Transformer blocks. As shown in Figure 1, this phase primarily involves large matrix-matrix multiplications (GEMMs), which are well-suited to the dense compute structures of modern accelerators.

**Decoding.** After the prefill phase, the model enters the decoding phase, where tokens are generated sequentially, one at a time. As shown in Figure 1, each iteration reuses the same model weights while processing only a single token, turning the computation into a matrix-vector multiplication (GEMV) with extremely limited parallelism and data reuse. Although both phases share the same



**Figure 2: Systolic array inefficiency under decoding ( $M=1$ ) for both output-stationary (left) and weight-stationary (right) dataflows.**

model architecture, prefill is compute-bound, whereas decoding is memory-bound and severely underutilizes compute resources [2, 34, 41].

**Takeaway:** Prefill produces large, highly parallel GEMMs, whereas decoding produces GEMMs whose token dimension collapses to  $M=1$ . This intrinsic mismatch leads to substantial efficiency gaps on conventional accelerators.

## 2.2 Rethinking System Scheduling

Modern LLM serving systems [34, 59, 61] introduce highly dynamic workloads driven by runtime scheduling, which continuously changes the balance between prefill and decoding. These dynamics expose a fundamental mismatch between modern LLM serving and static hardware execution models.

**Continuous Batching (CB).** CB [2, 34, 59] aggregates requests to improve decoding throughput, but yields highly variable GEMM shapes depending on the number of active tokens. Batch sizes fluctuate as requests arrive and complete, so computation shifts continuously between flat, memory-bound GEMVs and larger, more parallel workloads, leaving hardware utilization inconsistent.

**Prefill-Decoding Disaggregation (PDD).** To mitigate interference between phases, modern systems adopt PDD [41, 43, 62], which executes the two phases on separate devices and thereby removes the resource contention that degrades decoding throughput when they are co-located. While PDD improves performance, it also exposes a key limitation: statically deploying separate hardware for prefill and decoding leads to resource imbalance under dynamic workloads, as the ratio between phases continuously changes.

**Takeaway.** LLM serving exhibits highly dynamic workload characteristics, where both GEMM shapes and phase ratios vary over time. This requires a unified architecture that can efficiently support both phases and adapt to changing workloads. Such adaptivity must be coordinated with system scheduling, enabling joint system-architecture co-design across quantization, array organization, and execution scheduling.

## 2.3 Rethinking Array Architecture

Systolic-array-based accelerators [10, 28, 29, 32, 47] are widely used for LLM inference due to their high throughput and efficient data reuse. However, their efficiency relies on GEMM patterns, which break down under decoding GEMV and are further exacerbated by dynamic GEMM shapes introduced by CB.

**Table 1: Representative accelerators and their MEU adaptability. *DynaCore* figures are per core.**

| Architecture                                                  | MEU ( $m \times n$ )                      | Dims      | Adaptation | Target             |
|---------------------------------------------------------------|-------------------------------------------|-----------|------------|--------------------|
| <i>Reconfigurable arrays: conformal fission</i>               |                                           |           |            |                    |
| Planaria [19]                                                 | $128 \times 128 \rightarrow 32 \times 32$ | $m=n$     | Conformal  | Multi-tenant DNN   |
| SARA [46]                                                     | $128 \times 128 \rightarrow 4 \times 4$   | $m=n$     | Conformal  | Multi-DNN scaling  |
| <i>Fixed arrays: LLM quantization / sparsity accelerators</i> |                                           |           |            |                    |
| Olive [21]                                                    | $32 \times 32$                            | —         | Fixed      | Prefill GEMM       |
| BitVert [9]                                                   | $16 \times 32$                            | —         | Fixed      | Prefill GEMM       |
| FIGNA [26]                                                    | $128 \times 128$                          | —         | Fixed      | Prefill GEMM       |
| FIGLUT [39]                                                   | $64 \times 64$                            | —         | Fixed      | Prefill GEMM       |
| TransArray [22]                                               | $192 \times 256$                          | —         | Fixed      | Prefill GEMM       |
| <i>Reconfigurable arrays: asymmetric reshaping</i>            |                                           |           |            |                    |
| <i>DynaCore</i> (Ours)                                        | $32 \times 32 \rightarrow 1 \times 1024$  | $m, n, k$ | Asymmetric | Prefill + Decoding |

**Notation.** Throughout this paper, uppercase  $M \times N \times K$  denotes the full dimensions of a GEMM as issued by the model, while lowercase  $m \times n \times k$  denotes the tile the hardware executes as one unit.

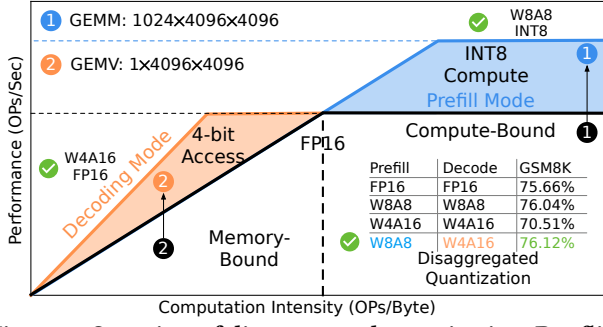
**Minimum Efficient Unit.** To analyze the efficiency limits of such architectures under LLM workloads, we introduce the **Minimum Efficient Unit (MEU)**: the smallest GEMM tile that fully utilizes both the compute array and its on-chip data movement. An MEU is inherently **three-dimensional**,  $\text{MEU} = m \times n \times k$ . The spatial extents  $m$  and  $n$  are set by the physical array organization and bound the output tile produced in parallel. The third extent  $k$  is temporal: it is the reduction depth the array must consume before every PE becomes active. Unlike  $m$  and  $n$ ,  $k$  is not fixed by the array dimensions alone; it depends on how operands are delivered and propagated, and must be determined per implementation. Utilization for a GEMM of shape  $M \times N \times K$  is therefore

$$U = \underbrace{\frac{\min(M, m) \cdot \min(N, n)}{m \cdot n}}_{\text{spatial}} \cdot \underbrace{\frac{K}{K + k}}_{\text{temporal}}. \quad (1)$$

Full utilization requires  $M \geq m$ ,  $N \geq n$ , and  $K \gg k$  simultaneously. We call a GEMM meeting all three conditions a **tall GEMM**, and one with  $M < m$  a **flat GEMM**. Decoding produces flat GEMMs: with  $M=1$  on an  $S \times S$  array the spatial term collapses to  $1/S$ .

**Systolic Array Limitation.** Modern LLM accelerators [21, 22, 26, 39] predominantly adopt 2D systolic arrays to exploit data reuse across both spatial dimensions, amortizing on-chip buffer bandwidth. Table 1 summarizes representative designs and the MEU each of them realizes. However, as illustrated in Figure 2, decoding fundamentally breaks this reuse pattern. With only a single input token ( $M=1$ ), activations cannot be reused across rows, and weights cannot be efficiently shared across the array. As a result, reuse collapses, leaving most processing elements idle. This reveals a fundamental mismatch: systolic arrays are designed to trade on-chip bandwidth for data reuse, while decoding workloads provide little opportunity for reuse. Consequently, existing designs fail to efficiently support decoding, despite being well optimized for prefill.

**Prior Reconfigurable Arrays.** Reconfigurable arrays are not new, but existing designs reshape only the spatial pair  $(m, n)$ , and do so *symmetrically*. Planaria [19] fissions a large array into fixed square sub-arrays so that concurrent DNNs can be co-located;



**Figure 3: Overview of disaggregated quantization. Roofline analysis illustrates the contrast between GEMM (prefill) and GEMV (decoding). Bottom-right: accuracy comparison on LLaMA3-8B evaluated on GSM8K.**

SARA [46] learns flexible GEMM configurations and scales a monolithic array down to finer square cells to match varying layer dimensions. Both target *multi-tenant* or *multi-DNN* execution, where each tenant still presents a well-formed 2D tile, and both therefore traverse only the conformal diagonal  $m = n$ , as listed in Table 1. This constraint, not the granularity of the fission, is what excludes decoding. Conformal splitting shrinks  $m$  and  $n$  together, whereas a flat GEMM requires them to move in *opposite* directions:  $m$  must fall toward one while  $n$  grows to absorb the freed PEs. No amount of finer conformal subdivision reaches this operating point, which is why Planaria and SARA saturate at MEU floors of  $32 \times 32$  and  $4 \times 4$  respectively.

**LLM-Related Accelerators.** Recent LLM accelerators [8, 9, 21–23, 25, 26, 39] improve efficiency through quantization, sparsity, or lookup-based computation, but keep a fixed array organization throughout execution. Their MEUs, listed in Table 1, are static and in several cases far larger than a decoding GEMV. While these techniques reduce arithmetic and memory cost, they all assume efficient data reuse and leave the array shape untouched, so none addresses decoding underutilization.

**Takeaway.** Decoding is not merely a *small* workload; it is *dimensionally mismatched*. This motivates a *splittable systolic array* that reshapes its MEU across space and time and adapts its operand paths asymmetrically, listed in Table 1.

## 2.4 Rethinking Quantization Strategies

Quantization [4, 16, 18, 31, 36, 37, 48, 57] improves LLM inference efficiency by reducing the precision of weights and activations. Existing approaches typically adopt a unified configuration across both phases, as reflected in prior accelerators in Table 1. However, as the roofline in Figure 3 shows, prefill and decoding sit on opposite sides of the ridge point, making a single scheme suboptimal.

**Decoding.** Operating in the memory-bound region, decoding is dominated by weight traffic, so weight-only quantization [31, 36] (e.g., W4A16) reduces the memory-bound latency floor in proportion to the weight bit-width. Activations, whose traffic is negligible, are best left in FP16: this preserves accuracy at little memory cost, whereas dual-side quantization [4, 37, 57] (e.g., W8A8) raises weight precision and thus traffic. Reducing compute precision brings little benefit either, since decoding demands far less arithmetic than FP16 or INT8 units can supply. This gain, however, materializes

only if the compute array keeps pace with the memory system. A GEMV on an  $S \times S$  output-stationary array activates a single row, leaving  $1/S$  of the PEs busy; on a  $128 \times 128$  array at 500 MHz with 4 TB/s of bandwidth, the resulting compute time exceeds the FP16 weight-streaming time by about  $31\times$ , regardless of model size. Decoding on such an array is therefore utilization-bound rather than bandwidth-bound, and weight-only quantization pays off only after array utilization is restored (Section 4).

**Prefill.** For long prompts, prefill sits far from the memory-bound region, so reducing memory traffic yields little. Here throughput is the constraint, and dual-side quantization (e.g., W8A8) is the better fit, doubling arithmetic throughput through INT8 units while maintaining model accuracy. Prefill GEMMs are tall only when the prompt is long, however: since  $M$  equals the number of prompt tokens, a short prompt yields  $M$  below the array dimension and inherits the same row underutilization as decoding.

**No Duplication Cost.** Assigning different precisions to the two phases does not imply storing two copies of the model. Under PDD, prefill and decoding already execute on separate devices, each holding only the weights for its own phase, so the W8A8 and W4A16 copies reside where they are used and neither device carries the other’s format.

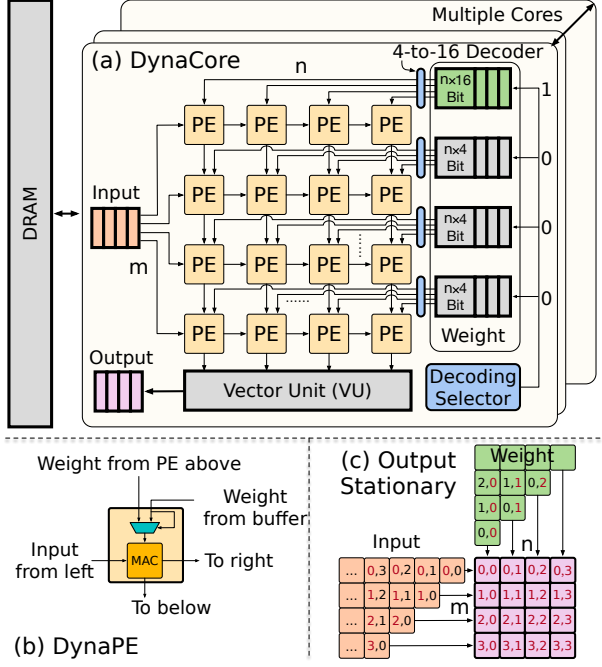
**Accuracy Gain.** As shown in Figure 3 (bottom-right), weight-only quantization (e.g., W4A16) leads to noticeable accuracy degradation. In contrast, disaggregated quantization, using W4A16 for decoding and W8A8 for prefill, achieves near-lossless accuracy while maintaining high efficiency. Notably, this configuration outperforms both standalone W8A8 and W4A16, achieving a superior efficiency–accuracy trade-off.

**Takeaway.** *Disaggregated quantization* lowers precision on exactly the path that asymmetric reshaping stresses, leaving activations untouched on the path that needs no additional bandwidth. Supporting both precisions in one datapath, however, requires a mixed-precision design aware of the array shape, one that aligns with the temporal dimension  $k$  as well as with the asymmetric provisioning of operand paths.

## 3 DynaCore Architecture

*DynaCore* is organized as a set of identical cores, each an output-stationary systolic array with its own input, weight, and output buffers, as shown in Figure 4(a). A core is the unit at which an MEU is realized, and cores reshape independently, so a device can run several MEU configurations concurrently.

Within a core, reconfiguration proceeds along two axes. *Spatially*, the array trades  $m$  against  $n$  at a fixed PE count. The mechanism is the selector inside each *DynaPE* (Figure 4(b)), which chooses whether the PE receives its weight from the PE above or directly from the buffer. Rows fed from the buffer begin a new logical row of the reshaped array, so the selector pattern alone determines the  $m \times n$  organization, leaving the array topology untouched. Because only the weight side changes while the input side continues to broadcast a single stream, the two operand paths are provisioned asymmetrically rather than scaled together. *Temporally*, the reduction dimension becomes mappable: the array partitions a long reduction across PE groups and folds their partial sums in place,



**Figure 4: Overview of the DynaCore architecture. (a) Multiple identical cores, each a splittable systolic array. (b) DynaPE selects its weight source between the PE above and the buffer. (c) Output-stationary dataflow.**

which the output-stationary dataflow (Figure 4(c)) supports without widening the output path.

The remainder of this section follows the two axes in turn. Section 4 covers spatial reshaping, from tall GEMMs where a square configuration suffices to flat GEMMs where the weight and input paths must be provisioned asymmetrically. Section 5 adds the temporal axis through Split-K. Section 6 then shows how disaggregated quantization matches all three dimensions: it supplies the weight bandwidth that asymmetric reshaping of  $m$  and  $n$  demands, and its mixed-precision datapath directs the throughput freed at low precision along  $K$ .

## 4 Asymmetric Spatial Reshaping

This section describes how a core reshapes the spatial extents of its MEU. The PE count is fixed at  $S^2$ , so  $m$  and  $n$  can only be traded against each other; what varies is how the trade is made and what it costs on each operand path.

### 4.1 Selector-Based Reshaping

Each DynaPE chooses its weight source between the PE above and the weight buffer (Figure 4(b)). A PE fed from the buffer starts a new weight stream; one fed from above continues and reuses the stream of the row above it. The selector pattern therefore partitions the array into groups, each forming one logical row of the reshaped array, so a pattern with  $g$  asserted rows yields

$$m = S/g, \quad n = gS, \quad m \cdot n = S^2. \quad (2)$$

Reshaping rewrites  $S$  control bits rather than the interconnect, leaving the array topology unchanged, and its cost falls on one side

only:  $g$  groups require  $g$  concurrent weight streams while the input side carries a single broadcast stream.

### 4.2 Tall GEMM: Splitting $M$ and $N$

A tall GEMM exceeds the core in both spatial dimensions, so it is tiled along  $M$  and  $N$  and each tile maps directly onto the array. Figure 5(a) shows the resulting configuration on a  $4 \times 4$  core with a single asserted row. Weights enter at the top and propagate downward, reused across all four rows; inputs propagate horizontally, reused across all four columns. With  $M \geq m$  and  $N \geq n$ , the spatial term of Equation 1 reaches unity, and reuse in both directions holds on-chip traffic at its minimum of  $m+n$  operands per cycle for  $m \cdot n$  PEs. The temporal extent follows from the same mapping: an operand entering the array reaches the farthest PE after  $\max(m, n)$  cycles, so the array saturates in four cycles, giving  $\text{MEU} = 4 \times 4 \times 4$ .

### 4.3 Flat GEMM: Asymmetric Regrouping

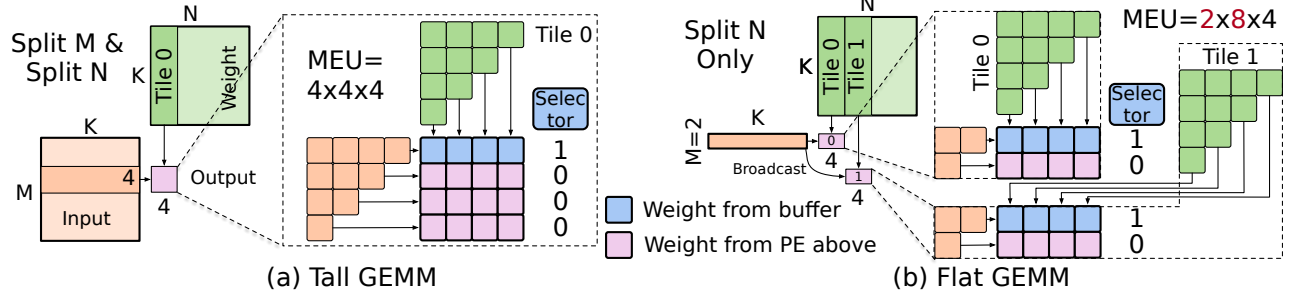
**Reshaping the MEU.** A flat GEMM leaves  $M$  below the array dimension, so the tiling reduces to splitting  $N$  alone: there is nothing left to split along  $M$ , and a square mapping simply wastes the rows it cannot fill. The core instead reduces  $m$  to the available token count and regroupes the freed rows into wider logical rows along  $N$ . Figure 5(b) shows  $M=2$  on the same core: two asserted rows, giving  $m=2$  and  $n=8$ . Although  $m$  has halved, the temporal extent is unaffected. Each group still spans four columns of the physical array, so an operand still reaches the farthest PE after  $\max(m, n) = 4$  cycles within its group, leaving  $k=4$  and  $\text{MEU} = 2 \times 8 \times 4$ .

**Asymmetric Adaptation.** The two operand paths move in opposite directions under this regrouping. Halving  $m$  removes half of the vertical weight reuse, so the buffer must issue two independent weight streams instead of one; in general,  $g$  groups require  $g$  streams. The input path is untouched, since the same tokens serve every tile and are simply broadcast. This asymmetry is intrinsic to LLM decoding rather than an artifact of the mapping: with  $M$  small there is little activation data to begin with, so widening the input path buys nothing, and all of the pressure falls on weight delivery. It is the same observation that makes weight-only quantization the prevailing choice for decoding, and Section 6 shows how the two meet.

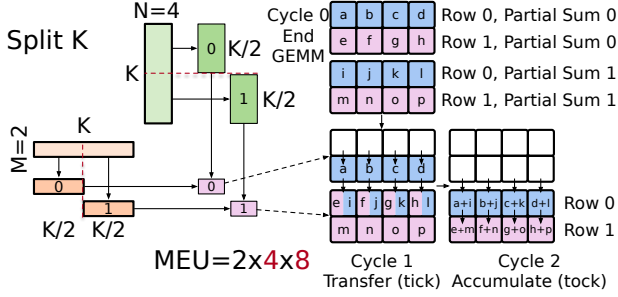
**Continuum of Configurations.** The square and flattened configurations are endpoints of a continuum rather than separate designs. Any  $g$  dividing  $S$  is a valid selector pattern, so by Equation 2 an  $S \times S$  core realizes  $S \times S, \frac{S}{2} \times 2S, \dots, 1 \times S^2$  at the same PE count; on a  $32 \times 32$  core this gives six reachable shapes between fully square and fully flattened. The temporal extent is invariant across all of them, since propagation is bounded by the physical array dimension regardless of how rows are grouped. The array therefore tracks the token count of the current batch, which matters under continuous batching where that count changes from iteration to iteration.

## 5 Temporal Split-K Mapping

Spatial regrouping converts rows freed from  $m$  into a wider  $n$ , which requires the workload to supply enough columns to occupy them. At  $M=1$  a  $32 \times 32$  core flattens to  $1 \times 1024$ , and a device instantiating



**Figure 5: Spatial reshaping in DynaCore.** (a) A tall GEMM is split along both  $M$  and  $N$ , and the core runs as a square array. (b) A flat GEMM is split along  $N$  only; the core halves  $m$  and doubles  $n$ , broadcasting the input while doubling weight delivery.



**Figure 6: Temporal split- $k$  mapping.** The array is partitioned into  $t = 2$  groups along  $K$ , each producing a partial sum for the same output tile.

sixteen such cores needs more still. Many decoding GEMMs fall short; we call a shape with  $M < m$  and  $N < n$  a *narrow and flat* GEMM. Here no selector pattern helps: shrinking  $n$  to match  $N$  leaves the rows beyond the first few idle, while keeping  $n$  large leaves their columns empty. What remains is  $K$ , which stays large in exactly these cases. Mapping part of that reduction onto the array is therefore the only way to recover utilization, and it is the third dimension of the MEU introduced in Section 2.3.

### 5.1 Partitioning the Reduction

The array is divided into  $t$  groups along  $K$ , each assigned a contiguous slice of the reduction and each producing an independent partial sum for the *same* output tile. The PE budget is then shared three ways,

$$m \cdot n \cdot t = S^2, \quad k = t \cdot \max(m, n), \quad (3)$$

so the temporal extent grows with the number of groups: the core consumes  $t$  slices of depth  $\max(m, n)$  before every PE has contributed. Figure 6 shows  $M=2$ ,  $N=4$  on a core that would otherwise flatten to  $2 \times 8$ . Since  $N=4$  cannot fill eight columns, the configuration uses  $m=2$ ,  $n=4$ , and  $t=2$  instead, giving  $\text{MEU} = 2 \times 4 \times 8$  at the same PE count.

Physically nothing new is required for the compute phase. Each group is an ordinary  $m \times n$  sub-array executing an ordinary output-stationary GEMM over its own slice of  $K/t$ , driven by the same selector pattern used for spatial regrouping. The weight buffer supplies  $t$  streams exactly as it would for  $t$  spatial groups, so the delivery path is unchanged.

### 5.2 Tick-Tock Reduction

Splitting along  $K$  differs from splitting along  $M$  or  $N$  in one respect that shapes the entire design. Output tiles produced by different  $m$  or  $n$  groups are independent and simply drain in parallel. Partial sums produced by different  $k$  groups are not: they are addends of the same result and must be reduced before leaving the array.

A dedicated adder tree is the obvious solution and an impractical one. Reducing across  $t$  groups spanning the array height requires a tree for every one of the  $n$  columns, together with wiring that gathers operands from PEs at arbitrary vertical distances. At array scale this approaches the cost of a second PE array, and its routing is precisely the long-distance, many-to-one interconnect that dominates physical design.

*DynaCore* instead reduces along the vertical path the array already has. The final group is held *stationary* and serves as the destination accumulator, while every other group releases its partial sums downward one row per cycle. A *tick* moves a partial sum one row down without accumulating; a *tock* adds the arriving value into the local accumulator, which happens once it reaches its counterpart in the group below. A logical row of width  $n$  occupies  $n/S$  physical rows and a group holds  $m$  of them, so one group spans

$$h = \frac{m \cdot n}{S} = \frac{S}{t} \quad (4)$$

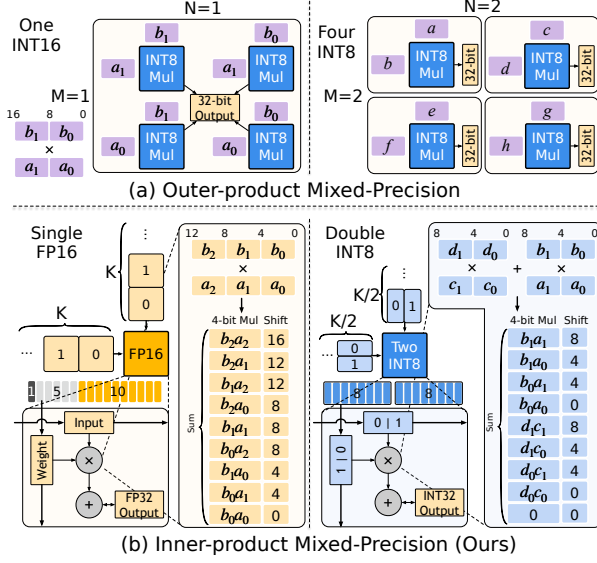
physical rows, using  $m \cdot n \cdot t = S^2$  from Equation 3. In Figure 6,  $t=2$  on a  $4 \times 4$  core gives  $h=2$ : one tick and one tock.

### 5.3 Cost at the Output Path

The scheme is inexpensive because it is the array's existing drain, with one change. In the baseline design, every PE holds a result and the array empties row by row, each value passing downward through the PEs beneath it, which takes  $S$  cycles. Under Split- $K$  the same  $S$  cycles are spent, but the values passing through the final group are added into it rather than forwarded past it. Reduction and evacuation are thus the same phase: the array still clears in  $S$  cycles, and what leaves it is the reduced result rather than  $t$  separate partial sums.

## 6 Disaggregated Quantization

Prefill is compute-bound and benefits from dual-side quantization (W8A8), while decoding is memory-bound and benefits from weight-only quantization (W4A16). Serving both on one substrate requires PEs that execute either without duplicating hardware. This



**Figure 7: Mixed-precision construction. (a) Outer-product schemes expand the output tile from  $1 \times 1$  to  $2 \times 2$ . (b) Ours packs operands along  $K$ , leaving the output tile at  $1 \times 1$  in both modes.**

section shows that the way this is done matters as much as the fact that it is done: the conventional construction inflates the output path, whereas ours leaves it untouched and, in doing so, matches both the asymmetric regrouping of Section 4 and the Split- $K$  mapping of Section 5.

### 6.1 Outer-Product versus Inner-Product Construction

Bit-composable PEs [23, 49] build a high-precision multiply from low-precision sub-multipliers, and at reduced precision re-task those sub-multipliers to carry more work. The question is where that work goes.

**Outer product.** In the conventional choice (Figure 7(a)), one INT16 multiply decomposes into the four cross terms of  $\{a_1, a_0\} \otimes \{b_1, b_0\}$ , shifted and summed into a single output; at INT8 the same four multipliers are re-interpreted as four *unrelated* operand pairs that share no reduction. Throughput quadruples, but the output tile expands from  $1 \times 1$  to  $2 \times 2$ , so the PE needs four accumulators instead of one. Our synthesis results show that this construction costs over  $2 \times$  the area of a plain INT16 unit at equal multiplier count, since the added accumulators and output registers, not the multipliers, dominate. The penalty also compounds with the floating-point path: with four independent adders already committed to separate outputs, none can be folded into an FP16 accumulation, so FP16 support would require a datapath of its own rather than reusing what is there. Together, these costs make the construction impractical.

**Inner product.** *DynaCore* instead assigns the freed sub-multipliers to operands *adjacent along  $K$* , which the GEMM already reduces. An FP16 multiply needs an 11-bit mantissa product, implemented as a 12-bit multiplier built from nine 4-bit sub-multipliers combined by shift-and-add; two INT8 multipliers need eight of those same

nine (Figure 7(b)). The INT8 mode therefore reuses the FP16 structure almost entirely, packing a pair of  $K$ -adjacent operands into the 16-bit input word and summing their products in place, exactly as the output-stationary dataflow would have summed them across two cycles. The PE still holds one accumulator and emits one 32-bit result; only the effective depth consumed per cycle changes with precision. Operand widths at the array boundary are unchanged, so nothing about the mapping, the drain, or the interconnect is affected.

### 6.2 Matching the Array Design

**Asymmetric regrouping.** Flattening the array multiplies the weight bandwidth required while leaving the input path at a single broadcast stream, and weight-only quantization relieves exactly that side: at 4 bits the same buffer port supplies four times as many elements per cycle, while activations stay in FP16 on the path that needed no extra bandwidth. Keeping activations at FP16 precision also confines dequantization to the buffer interface, so no dequantization logic enters the PEs even when all  $S^2$  of them are fed independently.

**Split- $K$  mapping.** Split- $K$  and the inner-product datapath make the same choice at two levels: the former spends surplus PEs on the reduction dimension, the latter spends surplus multiplier capacity there. Both land where the GEMM already reduces, so the work is absorbed by existing accumulators rather than by new outputs. Precision therefore acts only on  $k$ : packing two INT8 operands per cycle doubles the reduction depth a PE covers, leaving  $m$ ,  $n$ , and  $t$  and the reduction schedule untouched.

**Summary.** A single datapath supports W8A8 prefill and W4A16 decoding by directing low-precision throughput along  $K$  rather than into additional outputs. Output width and count are invariant to precision; weight bandwidth falls by  $4 \times$  where the flattened configuration needs it most, and neither the array shape, the drain path, nor the reduction schedule is affected.

## 7 System-Architecture Co-Scheduling

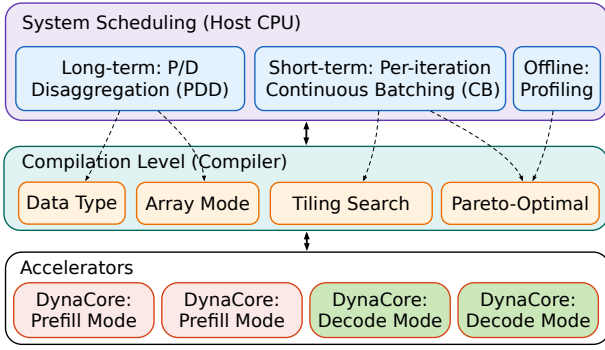
The reshaping mechanisms of Sections 4 and 5 expose a configuration space that must be navigated at runtime. *DynaCore* does so through three layers: a host *system scheduler*, a *compilation* layer that selects an MEU for each batch, and the *accelerators*. Figure 8 gives an overview. Prior work has studied system-level scheduling extensively, so we omit low-level serving heuristics and focus on the interaction between scheduling and the proposed architecture.

### 7.1 System Scheduling

The top layer runs on the host CPU and supports continuous batching (CB) and prefill-decoding disaggregation (PDD) over multiple devices through abstract interfaces to the backend accelerators. It operates at two time scales.

**Long-term scheduling.** PDD assigns each device a phase, and the optimal prefill/decoding device ratio varies with the workload: long-input traces favor prefill capacity, long-output traces favor decoding. *DynaCore* adapts this ratio from workload statistics and fixes the per-phase quantization setting accordingly, which we evaluate in Section 8.5.

**Short-term scheduling.** Within each iteration, CB determines which requests are batched together, and thus the token count that



**Figure 8: DynaCore three-layer co-scheduling.** The host scheduler handles long-term PDD and short-term CB; the compiler selects an MEU configuration from offline profiling; the accelerators execute under the selected configuration.

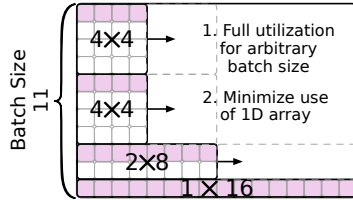
reaches the accelerator. This count is what the compilation layer consumes: it changes from iteration to iteration, and each value implies a different MEU.

## 7.2 Compilation Layer

Both control paths feed a compilation layer that maps the assigned batch onto concrete MEUs. Long-term scheduling fixes the data type; the compiler then chooses how to decompose the batch and which extents  $(m, n, t)$  to use for each part.

### Pyramid packing.

A batch size need not map onto a single MEU. When  $M$  falls between two reachable configurations, forcing one shape either idles rows or splits the batch across passes, so *DynaCore* instead decomposes the batch into several MEUs executed within the same scheduling window. Figure 9 shows  $M=11$  on a  $4 \times 4$  core decomposed into  $4 \times 4$ ,  $4 \times 4$ ,  $2 \times 8$ , and  $1 \times 16$ , filling the array exactly. The decomposition is hierarchical, taking the largest configuration that fits at each step, which keeps utilization high at arbitrary batch sizes while using the fully flattened configuration only for the remainder. The latter matters because flattening trades weight reuse for occupancy, so minimizing its share reduces memory traffic and energy.



**Figure 9: Pyramid packing:  $M=11$  split into four MEUs on a  $4 \times 4$  core.**

The decomposition is hierarchical, taking the largest configuration that fits at each step, which keeps utilization high at arbitrary batch sizes while using the fully flattened configuration only for the remainder. The latter matters because flattening trades weight reuse for occupancy, so minimizing its share reduces memory traffic and energy.

**Configuration search.** Packing fixes  $m$  for each part; the remaining freedom is how to divide the rest of the PE budget between  $n$  and  $t$ . Several choices reach comparable utilization while differing in memory behavior: a larger  $n$  covers more output columns per pass but consumes weight bandwidth, whereas a larger  $t$  maps more of the reduction onto the array and suits shapes whose  $N$  is bounded by the model. Total energy follows the same tension, since the wider configuration draws higher instantaneous power yet can finish sooner. The compiler therefore scores every candidate satisfying  $m \cdot n \cdot t = S^2$  against a joint latency–energy objective; Alg. 1 states the procedure.

### Algorithm 1: Batch decomposition and MEU selection

---

**Input:** shape  $(M, N, K)$ ; core dimension  $S$ ; energy weight  $\lambda$ ; profiling tables  $\mathcal{T}_{\text{lat}}, \mathcal{T}_{\text{eng}}$   
**Output:** list  $\mathcal{P}$  of split configurations  $(m, n, t)$

```

1  $\mathcal{P} \leftarrow \emptyset; \quad r \leftarrow M;$ 
2 while  $r > 0$  do
3    $m \leftarrow \max\{2^i \leq \min(r, S)\};$            // split  $M$ 
4    $c^* \leftarrow \infty;$ 
5   foreach  $t \in \{1, 2, 4, \dots, S^2/m\}$  do
6      $n \leftarrow S^2/(mt);$                        // split  $N$ 
7      $k \leftarrow t \cdot \max(m, n);$                  // split  $K$ 
8      $\tau \leftarrow \lceil N/n \rceil \cdot \lceil K/k \rceil;$          // passes
9      $c \leftarrow \tau \cdot (\mathcal{T}_{\text{lat}}[m, n, t] + \lambda \mathcal{T}_{\text{eng}}[m, n, t]);$ 
10    if  $c < c^*$  then
11       $c^* \leftarrow c; \quad (n^*, t^*) \leftarrow (n, t);$ 
12     $\mathcal{P} \leftarrow \mathcal{P} \cup \{(m, n^*, t^*)\}; \quad r \leftarrow r - m;$ 
13 return  $\mathcal{P};$ 
```

---

**Offline profiling.** Both tables are populated offline, once per core design, by measuring each reachable configuration on representative shapes;  $\lambda$  sets the latency–energy preference and is fixed per deployment. The search space is small, since  $m, n$ , and  $t$  are powers of two constrained by  $m \cdot n \cdot t = S^2$ , so a  $32 \times 32$  core admits only tens of candidates and the compiler enumerates them exhaustively. At runtime the result is a table lookup keyed by batch size rather than a search, which keeps the cost negligible relative to an iteration.

## 8 Evaluation

### 8.1 Methodology

**Models and Datasets.** We evaluate *DynaCore* on several modern LLMs, including Llama2-7B, Llama2-13B [54], and Llama3.1-8B [50]. We benchmark these models on ShareGPT [51] and Dolly [14] datasets, as well as production traces from MoonCake [43] (a conversation trace and a tool interaction trace), and a synthetic dataset with fixed input/output lengths. These datasets collectively represent diverse LLM inference scenarios with varying prompt lengths and generation lengths, covering both short-form and long-form conversational workloads. We adopt GPTQ [18] for disaggregated quantization, a widely used LLM quantization method, to quantize the model to W8A8 for prefill and W4A16 for decoding.

**Baseline Architectures.** We compare *DynaCore* against several representative GEMM architectures for DNN and LLM acceleration. These include BitFusion (BF) [49], a systolic architecture for low-precision DNNs, FIGLUT [39], an accelerator that leverages look-up tables for FP-INT GEMM in quantized LLMs, and Planaria [19] and SARA [46], reconfigurable systolic-array accelerators for DNN and LLM inference. In addition, we evaluate a systolic array with our proposed disaggregated quantization (DQ) for comparison. The detailed configurations of all architectures are summarized in the top part of Tbl. 2.

**System and Architecture Implementation.** We build a full-stack simulation framework that models both the frontend system

**Table 2: Comparison of baseline architectures and *DynaCore*. “Same” indicates the same configuration as prefill. Baselines use identical precision across prefill and decoding, while *DynaCore* adopts phase-specific configurations. All designs use 28nm, 500MHz, an 8MB buffer, and 4TB/s of DRAM bandwidth.**

| Architecture           | PE Array   |          | Precision |        |         | Area (mm <sup>2</sup> ) | Power (W) |
|------------------------|------------|----------|-----------|--------|---------|-------------------------|-----------|
|                        | Shape      | Unit     | Prefill   | Decode | KV      |                         |           |
| BitFusion [49]         | 128×128    | INT8 MAC | W8A8      | Same   | KV8-Q8  | 23.64                   | 12.89     |
| SA [33] w/ DQ          | 128×128    | DQ MAC   | W8A8      | W4A16  | KV4-Q16 | 65.09                   | 111.55    |
| FIGLUT [39]            | 4×(16×16)  | 16b LUT  | W4A16     | Same   | KV4-Q16 | 42.36                   | 89.26     |
| Planaria [19]          | 128×128    | INT8 MAC | W8A8      | Same   | KV8-Q8  | 70.98                   | 133.37    |
| SARA [46]              | 128×128    | INT8 MAC | W8A8      | Same   | KV8-Q8  | 68.83                   | 164.49    |
| <b><i>DynaCore</i></b> | 16×(32×32) | DQ MAC   | W8A8      | W4A16  | KV4-Q16 | 76.44                   | 115.74    |

**Table 3: *DynaCore* Setup**

|              |                                                                                                |
|--------------|------------------------------------------------------------------------------------------------|
| PE Array     | 16 × (32 × 32); Output Stationary                                                              |
| Vector Unit  | 16× (EXP ×32; MUL ×32; ADD ×32; SQRT ×1; DIV ×1)                                               |
| Quantization | Prefill: Input INT8, Weight INT8, KV INT8<br>Decoding: Input INT8/FP16, Weight 4-bit, KV 4-bit |
| Buffer       | Input 16×128KB/32Banks; Weight/Output 16×192KB/32Banks                                         |
| Memory       | HBM4E: 4.0 TB/s, 2,048-bit interface, 16 Gb/s/pin                                              |

behavior and the backend hardware execution. The frontend simulates an online LLM serving environment, supporting continuous batching and prefill–decoding disaggregation, enabling accurate microsecond-level serving dynamics. The backend implements a cycle-accurate simulator for *DynaCore* and all baseline architectures, capturing compute behavior and memory access under real LLM workloads.

We implement the *DynaCore* architecture in SystemVerilog and synthesize it using Synopsys Design Compiler with the TSMC 28 nm standard-cell library to obtain area overhead, leakage power, and dynamic power. The on-chip buffers are evaluated using CACTI 7.0 [5], while off-chip memory behavior and energy are modeled with DRAMSim3 [35] and integrated into our cycle-accurate simulator. The overall architectural setup is summarized in Table 3. For a fair comparison, all baseline accelerators are evaluated with the same methodology.

## 8.2 LLM Accuracy Comparison

In Tbl. 4, we evaluate our proposed *disaggregated quantization* method on the Llama3.1-8B-Instruct model. We compare our approach with weight-only quantization algorithm, GPTQ [18] (W4A16), as well as the dual-side quantization algorithms, Qserve [37] (W4A8) and GPTQ [18] (W8A8). Additionally, we include BCQ [40], which is supported by FIGLUT [39]. Our evaluations cover several downstream datasets, including ARC-Easy and ARC-Challenge [12], BoolQ [11], HellaSwag [60], PIQA [6], and GSM8K [13].

Overall, our *disaggregated quantization* method, which utilizes W8A8 for prefill and W4A16 for decoding, incurs an average accuracy drop of only **0.07%** compared to FP16. This drop is significantly less than that of GPTQ (W4A16), which shows a **2.81%** drop, and Qserve (W4A8), which demonstrates a drop of **1.51%**. Our method’s accuracy drop is close to that of GPTQ (W8A8), which is **0.09%**. These results illustrate that our approach achieves a balance

**Table 4: Accuracy comparison on LLaMA3.1-8B-Instruct across downstream tasks.**

| Method       | ARC-C        | ARC-E        | BoolQ        | HellaSwag    | PIQA         | GSM8K        |
|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| FP16         | 55.03        | 79.63        | 84.13        | 79.27        | 80.96        | 75.66        |
| W8A8-GPTQ    | 55.12        | 79.63        | 83.91        | <b>79.05</b> | 80.41        | 76.04        |
| W4A16-GPTQ   | 51.62        | 75.93        | 82.87        | 77.78        | 79.11        | 70.51        |
| W4A8-QServe  | 53.41        | 79.29        | 83.58        | 77.99        | 79.65        | 71.72        |
| W4A16-FIGLUT | –            | –            | –            | –            | –            | 72.38        |
| <b>Ours</b>  | <b>55.03</b> | <b>79.76</b> | <b>84.01</b> | 78.32        | <b>81.01</b> | <b>76.12</b> |

between accuracy and efficiency, preserving model quality while attaining high performance in both the prefill and decoding phases.

## 8.3 Architecture Performance Comparison

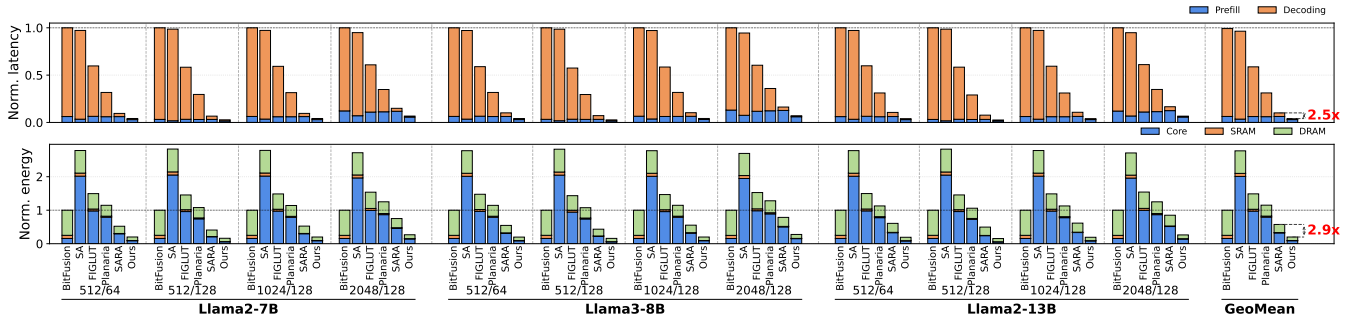
We evaluate *DynaCore* and all baseline architectures under a single-device setup to assess their standalone efficiency on LLM inference. Each workload consists of one input sequence with a typical input prompt and output generation length. We measure the end-to-end latency and energy normalized by BitFusion. The overall results are shown in Fig. 10.

**End-to-End Latency.** As shown in the top of Fig. 10, *DynaCore* achieves the lowest end-to-end latency among all baselines. Compared with the systolic array with DQ, *DynaCore* delivers a 23.07× speedup, primarily due to the decoding phase. By selecting the optimal hardware configuration from compilation, *DynaCore* maintains high utilization even at batch size 1, whereas the systolic array’s fixed  $128 \times 128 \times 128$  MEU leads to substantial underutilization.

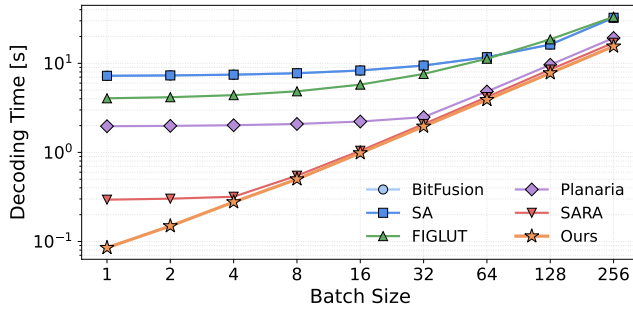
Compared with BitFusion and FIGLUT, *DynaCore* gains a 23.81× and 14.13× speedup, benefiting from the adaptive dataflow and higher INT8 throughput. We further compare against the reconfigurable systolic arrays Planaria and SARA, over which *DynaCore* achieves 7.57× and 2.48× end-to-end speedup. Although both reshape their arrays to improve decoding utilization, their coarse reconfiguration granularity ( $32 \times 32$  fission for Planaria and  $4 \times 4$  cells for SARA) still leaves the array underutilized on batch-1 GEMV. With finer-grained reconfiguration together with split-K, *DynaCore* provides a 23.03× and 3.48× decoding-phase speedup over Planaria and SARA, respectively.

**Energy.** The bottom of Fig. 10 shows the energy breakdown for all architectures. *DynaCore* achieves the best energy efficiency, outperforming BitFusion, Systolic Array, FIGLUT, Planaria, and SARA by 4.96×, 13.76×, 7.39×, 5.71×, and 2.86×, respectively. These gains mainly follow from the latency reductions discussed above. Although *DynaCore* has slightly higher on-chip power due to extra hardware support for splittable array and DQ, its shorter execution time reduces leakage energy and yields the highest overall efficiency.

**Hardware Overhead.** The area and on-chip power of *DynaCore* and all baselines are summarized in the last two columns of Tbl. 2. Compared to BitFusion INT8, the systolic array with DQ incurs higher area and power because its processing elements support both INT8 and FP16. *DynaCore* introduces further overhead over the systolic DQ design due to its shape-adaptive execution. Overall, these mechanisms introduce moderate overhead that is well justified by the latency improvements reported above.



**Figure 10: End-to-end latency and energy consumption comparison (lower is better). The numbers XX/YY shown below each group indicate the input and output lengths, respectively: XX denotes the prompt length, while YY denotes the generated output length.**



**Figure 11: Decoding Latency under differing batch sizes**

## 8.4 Batching Analysis

To further characterize *DynaCore* across different regimes, we evaluate the decoding latency of Llama-3-8B on a single device with batch sizes ranging from 1 to 256. As shown in Fig. 11, latency remains nearly constant while the PE array is underutilized and increases approximately linearly once the array becomes compute-bound.

The fixed  $128 \times 128$  systolic array reaches saturation at a batch size of approximately 128. The reconfigurable designs saturate earlier: Planaria at around 32, SARA at around 4, and *DynaCore* at around 1. *DynaCore* reaches high utilization even at small batch sizes because its fine-grained array partitioning and split- $K$  execution expose substantially more parallelism. Beyond saturation, *DynaCore* retains an advantage over the baselines because its compiler selects the most efficient configuration and scheduling strategy for each workload regime.

Consequently, *DynaCore* achieves the lowest latency across the entire batch-size range, reducing latency by 85.2 $\times$  over the systolic array at batch size 1 and by 2.09 $\times$  at batch size 256. Even at large batch size, it achieves the lowest 60.7 ms per-request latency, compared with 66.5 ms for SARA, 75.2 ms for Planaria, and 126.9 ms for the systolic array.

## 8.5 Multi-Device Evaluation for LLM Serving

Beyond single-device performance, we evaluate *DynaCore* in a multi-device online serving environment and focus on service-level objective (SLO) metrics that directly reflect user-perceived latency.

**Evaluation Setup.** We simulate an 8-device system handling incoming LLM requests. Each device is a chip instance of a given architecture with off-chip HBM, and devices are interconnected with 16 GB/s links, modeled following [3]; this inter-device communication cost is incurred whenever prefill and decoding are partitioned across separate devices. Unlike the single-device setting where prefill and decoding run on the same chip, here we follow the prefill-decoding disaggregation paradigm [41, 62], which improves SLO attainment. Several devices are assigned to prefill, and the remaining devices process decoding tasks.

The system is evaluated using the real-world datasets running Llama3.1-8B. For datasets without arrival timestamps, such as ShareGPT and Dolly, request arrivals follow a Poisson process. For MoonCake Conversation and MoonCake Tool, we use the real production timestamps. The scheduler uses a first-come-first-serve policy: prefill requests form batches according to arrival time, while decoding requests use continuous batching to improve utilization.

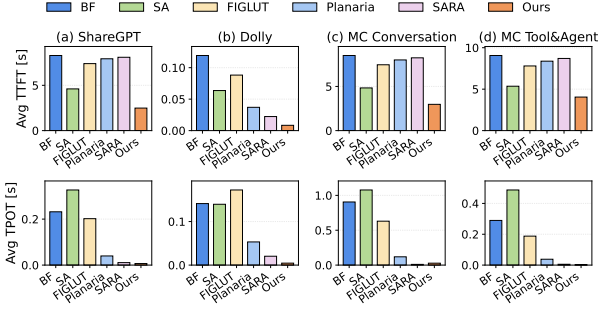
We measure two SLO metrics: Time to First Token (TTFT), which is mainly determined by prefill latency, and Time Per Output Token (TPOT), which reflects decoding performance.

**System-level Comparison.** We evaluate homogeneous multi-device systems in Fig. 12, using 4 prefill and 4 decoding devices for ShareGPT and Dolly, and 6 prefill and 2 decoding devices for MoonCake.

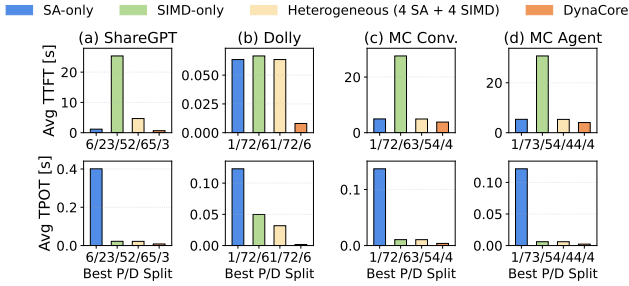
*DynaCore* achieves the lowest TTFT across all workloads. Baselines such as Planaria and FIGLUT remain prefill-bound because they do not accelerate large prefill GEMMs, whereas *DynaCore* combines disaggregated quantization with shape-adaptive execution. This yields a 2.97 $\times$  and 3.50 $\times$  average TTFT improvement over Planaria and FIGLUT, respectively.

*DynaCore* also achieves the best overall TPOT. Reconfigurable arrays such as Planaria are the strongest baselines, while FIGLUT is over an order of magnitude slower on average. For TPOT, *DynaCore* achieves an 8.02 $\times$  and 36.55 $\times$  average improvement over Planaria and FIGLUT, respectively. Its fine-grained splitting, Split- $K$ , and pyramid packing maintain high utilization for the small decoding batches common in online serving, delivering substantial SLO improvements across realistic workloads.

***DynaCore* compared with static system.** Fig. 13 compares *DynaCore* against three static eight-device systems: an SA-only design optimized for large- $M$  GEMMs, a SIMD-only design optimized



**Figure 12: Average TTFT and TPOT on real-world datasets. The Y-Axis values are in seconds.**



**Figure 13: Comparison of *DynaCore* with static homogeneous and heterogeneous multi-device systems.**

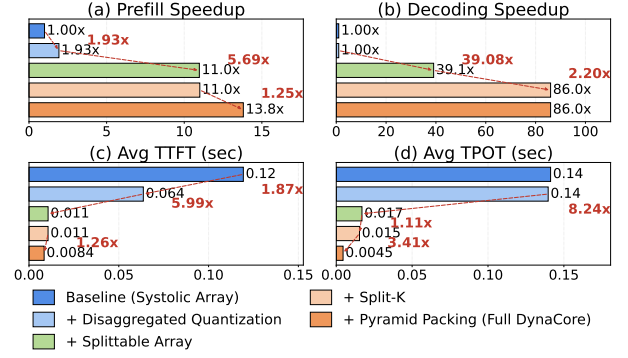
for small- $M$  GEMMs and GEMVs, and a heterogeneous system comprising four SA devices and four SIMD devices. Here, SIMD denotes a vector-style architecture that applies the same instruction across multiple lanes and is particularly effective for GEMV-dominated decoding. Each system contains the same total number of PEs as *DynaCore*.

The heterogeneous system serves as a strong baseline because it includes devices specialized for both prefill and decoding. However, its fixed 4:4 hardware partition becomes suboptimal when the ideal allocation between prefill and decoding devices varies across workloads. For example, on the decode-heavy Dolly workload, the heterogeneous design achieves a TPOT of 0.032 s, compared with only 0.0017 s for *DynaCore*. In contrast, *DynaCore* dynamically re-configures the same silicon resources at runtime, enabling it to match or outperform all static configurations in both TTFT and TPOT across all workload traces.

## 8.6 Ablation Study

We progressively add each *DynaCore* component to a vanilla 128×128 systolic array running Llama2-7B, and evaluate performance on the Dolly dataset, as shown in Fig. 14. Because the components are added cumulatively, each bar reports a component’s marginal gain given the components before it, and a component’s benefit can surface only once an earlier bottleneck is removed.

**Prefill.** For single-device inference at batch size 1 (Fig. 14(a)), disaggregated quantization, the splittable array, and pyramid packing improve prefill by 1.93×, 5.69×, and 1.25×, respectively. The 1.93×



**Figure 14: Ablation study of *DynaCore***

from DQ reflects the doubled INT8 throughput of W8A8. The large splittable-array gain stems from the short prompts of Dolly, which average only 17.9 tokens: with  $M$  far below the array dimension, prefill GEMMs are flat rather than tall, and the monolithic array suffers the same row underutilization as in decoding. For the long prompts in Fig. 10 (512–2048 tokens), prefill GEMMs are tall and the square configuration suffices, so the end-to-end gains there come almost entirely from decoding, consistent with Section 4.

**Decoding.** In decoding (Fig. 14(b)), DQ alone yields no speedup despite cutting weight traffic by 4×. On the vanilla array, a batch-1 GEMV activates one row of PEs, and the resulting compute time exceeds even the FP16 weight-streaming time by about 31× (Section 2.4); latency is bound by utilization, not bandwidth, so reducing traffic has no effect. The splittable array removes this bottleneck and delivers a 39.1× speedup, which in turn exposes the benefit of DQ: with FP16 weights, the bandwidth floor would cap the splittable-array gain at about 31×, so the excess is attributable to the 4-bit weights. Split-K adds another 2.20× by improving utilization for small GEMM shapes.

**Online serving.** In multi-device online serving (Fig. 14(c)(d)), the same components improve TTFT by 1.87×, 5.99×, and 1.26×, respectively. For TPOT, DQ again yields no gain on the vanilla array for the same reason, while the splittable array, Split-K, and pyramid packing provide improvements of 8.24×, 1.11×, and 3.41×. Pyramid packing matters far more here than at batch size 1, since continuous batching produces irregular  $M$  that a single MEU cannot tile efficiently.

## 9 Conclusion

We present *DynaCore*, a phase-adaptive architecture for efficient LLM serving. By co-designing array organization, quantization, and runtime scheduling, *DynaCore* aligns hardware execution with the distinct characteristics of prefill and decoding. Our design integrates a splittable systolic array, disaggregated quantization, and adaptive scheduling into a unified framework. Evaluation shows that *DynaCore* substantially reduces service-level latency over FIGLUT and Planaria, improving TTFT by up to 3.50× and 2.97× and TPOT by up to 36.55× and 8.02×, respectively, under real workloads. These results demonstrate the effectiveness of phase-aware system-architecture co-design for next-generation LLM serving.

## References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, Red Adila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brit-tany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jimoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Lukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Lukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McK-inney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O’Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael Pokorny, Michelle Pockross, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani San-turkar, Girish Sastry, Heateur Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vi-jayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, and Ramachandran Ramjee. 2023. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369* (2023).
- [3] Mohammad Alian, Krishna Parasuram Srinivasan, and Nam Sung Kim. 2018. Simulating PCI-Express interconnect for future system exploration. In *2018 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 168–178.
- [4] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L. Croci, Bo Li, Pashmina Cameron, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. 2024. Quarot: Outlier-free 4-bit inference in rotated llms. *Advances in Neural Information Processing Systems* 37 (2024), 100213–100240.
- [5] Rajeev Balasubramanian, Andrew B. Kahng, Naveen Muralimanohar, Ali Shafiee, and Vaishnav Srinivas. 2017. CACTI 7: New Tools for Interconnect Exploration in Innovative Off-Chip Memories. *ACM Trans. Archit. Code Optim.* 14, 2 (2017), 14:1–14:25. doi:10.1145/3085572
- [6] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. 2020. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34. 7432–7439.
- [7] Xinhua Chen, Sitao Huang, Cong Guo, Chiyue Wei, Yintao He, Jianyi Zhang, Hai Helen Li, and Yiran Chen. 2025. DPad: Efficient Diffusion Language Models with Suffix Dropout. *CoRR abs/2508.14148* (2025). arXiv:2508.14148 doi:10.48550/ARXIV.2508.14148
- [8] Yuzong Chen, Ahmed F. AbouElhamayed, Xilai Dai, Yang Wang, Marta Andronic, George A. Constantinides, and Mohamed S. Abdelfattah. 2025. BitMoD: Bit-serial Mixture-of-Datatype LLM Acceleration. In *IEEE International Symposium on High Performance Computer Architecture, HPCA 2025, Las Vegas, NV, USA, March 1–5, 2025*. IEEE, 1082–1097. doi:10.1109/HPCA61900.2025.00084
- [9] Yuzong Chen, Jian Meng, Jae-sun Seo, and Mohamed S Abdelfattah. 2024. BBS: Bi-directional bit-level sparsity for deep learning acceleration. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 551–564.
- [10] Yu-Hsin Chen, Tushar Krishna, Joel S Emer, and Vivienne Sze. 2016. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE journal of solid-state circuits* 52, 1 (2016), 127–138.
- [11] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. 2019. Boolq: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 conference of the north American chapter of the association for computational linguistics: Human language technologies, volume 1 (long and short papers)*. 2924–2936.
- [12] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457* (2018).
- [13] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168* (2021).
- [14] Mike Conover, Matt Hayes, Ankit Mathur, Jianwei Xie, Jun Wan, Sam Shah, Ali Ghodsi, Patrick Wendell, Matei Zaharia, and Reynold Xin. 2023. *Free Dolly: Introducing the World’s First Truly Open Instruction-Tuned LLM*. <https://www.databricks.com/blog/2023/04/12/dolly-first-open-commercially-viable-instruction-tuned-llm>
- [15] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *CoRR abs/2501.12948* (2025). arXiv:2501.12948 doi:10.48550/ARXIV.2501.12948
- [16] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. *Advances in neural information processing systems* 35 (2022), 30318–30332.
- [17] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *CoRR abs/1810.04805* (2018). arXiv:1810.04805 <http://arxiv.org/abs/1810.04805>
- [18] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2022. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. *CoRR abs/2210.17323* (2022). arXiv:2210.17323 doi:10.48550/ARXIV.2210.17323
- [19] Soroush Ghodrati, Byung Hoon Ahn, Joon Kyung Kim, Sean Kinzer, Brahmen-dra Reddy Yatham, Navateja Alla, Hardik Sharma, Mohammad Ali, Eiman Ebrahimi, Nam Sung Kim, Cliff Young, and Hadi Esmaeilzadeh. 2020. Planaria: Dynamic architecture fission for spatial multi-tenant acceleration of deep neural networks. In *2020 53rd Annual IEEE/ACM International Symposium on Microar-chitecture (MICRO)*. IEEE, 681–697.
- [20] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, Danny Wyatt, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Francisco Guzmán, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Govind That-tai, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jack Zhang, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelfer van der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasu-den Alwala, Karthik Prasad, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Kiritiika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Kushal Lakhotia, Lauren Rantala-Yeary, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Maria Tsim-poukelli, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kambadur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Ning Zhang, Olivier

Duchenne, Onur Çelebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohan Maheswari, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ronnie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia Kim, Sergey Edunov, Shaojiang Nie, Sharan Narang, Sharath Rapparth, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stephane Collot, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkrez, Vincent Gonguet, Virginie Do, Vish Vogeti, Vitor Albiero, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenyan Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiaofang Wang, Xiaoqing Ellen Tan, Xide Xia, Xinfeng Xie, Xuchao Jia, Xuewei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpierre Coudert, Zheng Yan, Zhengxing Chen, Zoe Papakipos, Aaditya Singh, Aayushi Srivastava, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gollati, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Amos Teo, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Dong, Annie Franco, Anuj Goyal, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardt, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Ce Liu, Changhan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Cynthia Gao, Damon Civin, Dana Beaty, Daniel Kreymer, Daniel Li, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkang Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Eric-Tuan Le, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Filippos Kokkinos, Firat Ozgenel, Francesco Caggioni, Frank Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Grant Herman, Grigory Sizov, Guangyi Zhang, Guna Lakshminarayanan, Hakan Inan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Hongyuan Zhan, Ibrahim Damlaj, Igor Molybog, Igor Tufanov, Ilias Leontiadis, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Janice Lam, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhie, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou U, Karan Saxena, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelena, Keqian Li, Kiran Jagadeesh, Kun Huang, Kunal Chawla, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Khabsa, Manav Avalani, Manish Bhatt, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Miao Liu, Michael L. Seltzer, Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vityatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikhil Mehta, Nikolay Pavlovich Laptev, Ning Dong, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanchandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Rangaprabhu Parthasarathy, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Wang, Russ Howes, Rutu Rinott, Sachin Mehta, Sachin Siby, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Mahajan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy Zha, Shishir Patil, Shiva Shankar, Shuqiang Zhang, Shuang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Summer Deng, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Koehler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihăilescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiaocheng Tang, Xiaojian Wu, Xiaolan Wang, Xilun Wu, Xinbo Gao, Yaniv Kleinman, Yanjun Chen, Ye Hu, Ye

- Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu Wang, Yu Zhao, Yuchen Hao, Yundi Qian, Yunlu Li, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, Zhiwei Zhao, and Zhiyu Ma. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [21] Cong Guo, Jiaming Tang, Weiming Hu, Jingwen Leng, Chen Zhang, Fan Yang, Yunxin Liu, Minyi Guo, and Yuhao Zhu. 2023. Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–15.
- [22] Cong Guo, Chiyue Wei, Jiaming Tang, Bowen Duan, Song Han, Hai Li, and Yiran Chen. 2025. Transitive Array: An Efficient GEMM Accelerator with Result Reuse. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 990–1004.
- [23] Cong Guo, Chen Zhang, Jingwen Leng, Zihan Liu, Fan Yang, Yunxin Liu, Minyi Guo, and Yuhao Zhu. 2022. ANT: Exploiting Adaptive Numerical Data Type for Low-bit Deep Neural Network Quantization. In *55th IEEE/ACM International Symposium on Microarchitecture, MICRO 2022, Chicago, IL, USA, October 1-5, 2022*. IEEE, 1414–1433. doi:10.1109/MICRO56248.2022.00095
- [24] Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi, Heyang Qin, Arash Bakhtiari, Lev Kurilenko, and Yuxiong He. 2024. DeepSpeed-fastgen: High-throughput text generation for LLMs via mii and DeepSpeed-Inference. *arXiv preprint arXiv:2401.08671* (2024).
- [25] Weiming Hu, Haoyan Zhang, Cong Guo, Yu Feng, Renyang Guan, Zhendong Hua, Zihan Liu, Yue Guan, Minyi Guo, and Jingwen Leng. 2025. M-ANT: Efficient Low-bit Group Quantization for LLMs via Mathematically Adaptive Numerical Type. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 1112–1126. doi:10.1109/HPCA61900.2025.00086
- [26] Jaeyong Jang, Yulhwa Kim, Juheun Lee, and Jae-Joon Kim. 2024. FIGNA: Integer Unit-Based Accelerator Design for FP-INT GEMM Preserving Numerical Accuracy. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 760–773. doi:10.1109/HPCA57654.2024.00064
- [27] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. *CoRR abs/2310.06825* (2023). arXiv:2310.06825 doi:10.48550/ARXIV.2310.06825
- [28] Norman P. Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Cliff Young, Xiang Zhou, Zongwei Zhou, and David Patterson. 2023. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th annual international symposium on computer architecture*. 1–14.
- [29] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, Rick Boyle, Pierre-luc Cantin, Clifford Chao, Chris Clark, Jeremy Coriell, Mike Daley, Matt Dau, Jeffrey Dean, Ben Gelb, Tara Vazir Ghaemmaghami, Rajendra Gottipati, William Gulland, Robert Hagmann, C. Richard Ho, Doug Hogberg, John Hu, Robert Hundt, Dan Hurt, Julian Ibarz, Aaron Jaffey, Alek Jaworski, Alexander Kaplan, Harshit Khaitan, Andy Koch, Naveen Kumar, Steve Lacy, James Laudon, James Law, Diemth Le, Chris Leary, Zhuyuan Liu, Kyle Lucke, Alan Lundin, Gordon MacKean, Adriana Maggiore, Maire Mahony, Kieran Miller, Rahul Nagarajan, Ravi Narayanaswami, Ray Ni, Kathy Nix, Thomas Norrie, Mark Omernick, Narayana Penukonda, Andy Phelps, Jonathan Ross, Matt Ross, Amir Salek, Emad Samadiani, Chris Severn, Gregory Sizikov, Matthew Snelham, Jed Souter, Dan Steinberg, Andy Swing, Mercedes Tan, Gregory Thorson, Bo Tian, Horia Toma, Erick Tuttle, Vijay Vasudevan, Richard Walter, Walter Wang, Eric Wilcox, and Doe Hyun Yoon. 2017. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th annual international symposium on computer architecture*. 1–12.
- [30] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. 2020. Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, 5156–5165. <http://proceedings.mlr.press/v119/katharopoulos20a.html>
- [31] Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W Mahoney, and Kurt Keutzer. 2023. Squeezellm: Dense-and-sparse quantization. *arXiv preprint arXiv:2306.07629* (2023).
- [32] Hsiang-Tsung Kung. 1982. Why systolic architecture? (1982).
- [33] Hsiang-Tsung Kung and Charles E Leiserson. 1979. Systolic arrays (for VLSI). In *Sparse Matrix Proceedings 1978, Vol. 1*. Society for industrial and applied mathematics Philadelphia, PA, USA, 256–282.
- [34] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.

- [35] Shang Li, Zhiyuan Yang, Dhiraj Reddy, Ankur Srivastava, and Bruce L. Jacob. 2020. DRAMsim3: A Cycle-Accurate, Thermal-Capable DRAM Simulator. *IEEE Comput. Archit. Lett.* 19, 2 (2020), 110–113. doi:10.1109/LCA.2020.2973991
- [36] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of machine learning and systems* 6 (2024), 87–100.
- [37] Yujun Lin, Haotian Tang, Shang Yang, Zhekai Zhang, Guangxuan Xiao, Chuang Gan, and Song Han. 2024. Qserve: W4a8kv4 quantization and system co-design for efficient llm serving. *arXiv preprint arXiv:2405.04532* (2024).
- [38] OpenAI. 2023. GPT-4 Technical Report. <https://cdn.openai.com/papers/gpt-4.pdf>. Accessed: 2025-11-16.
- [39] Gunho Park, Hyeokjun Kwon, Jiwoo Kim, Jeongin Bae, Baeseong Park, Dongsoo Lee, and Youngjoo Lee. 2025. FIGLUT: An Energy-Efficient Accelerator Design for FP-INT GEMM Using Look-Up Tables. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 1098–1111. doi:10.1109/HPCA61900.2025.00085
- [40] Seungcheol Park, Jeongin Bae, Beomseok Kwon, Minjun Kim, Byeongwook Kim, Se Jung Kwon, U Kang, and Dongsoo Lee. 2025. Unifying uniform and binary-coding quantization for accurate compression of large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 28468–28488.
- [41] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Ñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *51st ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2024, Buenos Aires, Argentina, June 29 - July 3, 2024*. IEEE, 118–132. doi:10.1109/ISCA59077.2024.00019
- [42] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. Efficiently scaling transformer inference. *Proceedings of machine learning and systems* 5 (2023), 606–624.
- [43] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Heyi Tang, Feng Ren, Teng Ma, Shangming Cai, Yineng Zhang, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2024. Mooncake: A kvcache-centric disaggregated architecture for llm serving. *ACM Transactions on Storage* (2024).
- [44] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving language understanding by generative pre-training. (2018).
- [45] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language Models are Unsupervised Multitask Learners. (2019).
- [46] Ananda Samajdar, Eric Qin, Michael Pellauer, and Tushar Krishna. 2022. Self adaptive reconfigurable arrays (sara) learning flexible gemm accelerator configuration and mapping-space using ml. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*. 583–588.
- [47] Ananda Samajdar, Yuhao Zhu, Paul Whatmough, Matthew Mattina, and Tushar Krishna. 2018. SCALE-Sim: Systolic CNN Accelerator Simulator. *arXiv preprint arXiv:1811.02883* (2018).
- [48] Wenqi Shao, Mengzhao Chen, Zhaoyang Zhang, Peng Xu, Lirui Zhao, Zhiqian Li, Kaipeng Zhang, Peng Gao, Yu Qiao, and Ping Luo. 2023. Omniquant: Omni-directionally calibrated quantization for large language models. *arXiv preprint arXiv:2308.13137* (2023).
- [49] Hardik Sharma, Jongse Park, Naveen Suda, Liangzhen Lai, Benson Chau, Vikas Chandra, and Hadi Esmaeilzadeh. 2018. Bit Fusion: Bit-Level Dynamically Composable Architecture for Accelerating Deep Neural Network. In *45th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2018, Los Angeles, CA, USA, June 1-6, 2018*, Murali Annamalai, Timothy Mark Pinkston, and Babak Falsafi (Eds.). IEEE Computer Society, 764–775. doi:10.1109/ISCA.2018.00069
- [50] Llama Team. 2024. The Llama 3 Herd of Models. *CoRR abs/2407.21783* (2024). arXiv:2407.21783 doi:10.48550/ARXIV.2407.21783
- [51] ShareGPT Team. 2023. ShareGPT. <https://sharegpt.com/>.
- [52] Shailja Thakur, Baleegh Ahmad, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. 2024. VeriGen: A Large Language Model for Verilog Code Generation. *ACM Trans. Des. Autom. Electron. Syst.* 29, 3, Article 46 (April 2024), 31 pages. doi:10.1145/3643681
- [53] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. *CoRR abs/2302.13971* (2023). arXiv:2302.13971 doi:10.48550/ARXIV.2302.13971
- [54] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madsen Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rishi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *CoRR abs/2307.09288* (2023). arXiv:2307.09288 doi:10.48550/ARXIV.2307.09288
- [55] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (Eds.). 5998–6008. <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- [56] Sinong Wang, Belinda Z. Li, Madsen Khabsa, Han Fang, and Hao Ma. 2020. Linformer: Self-Attention with Linear Complexity. *CoRR abs/2006.04768* (2020). arXiv:2006.04768 <https://arxiv.org/abs/2006.04768>
- [57] Guangxuan Xiao, Ji Lin, Mickaël Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. In *International Conference on Machine Learning, ICLR 2023, 23-29 July 2023, Honolulu, Hawaii, USA (Proceedings of Machine Learning Research, Vol. 202)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, 38087–38099. <https://proceedings.mlr.press/v202/xiao23c.html>
- [58] Chenwei Xiong, Cheng Liu, Huawei Li, and Xiaowei Li. 2025. HSLPilot: LLM-based High-Level Synthesis. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design (Newark Liberty International Airport Marriott, New York, NY, USA) (ICCAD '24)*. Association for Computing Machinery, New York, NY, USA, Article 226, 9 pages. doi:10.1145/3676536.3676781
- [59] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2022, Carlsbad, CA, USA, July 11-13, 2022*, Marcos K. Aguilera and Hakim Weatherspoon (Eds.). USENIX Association, 521–538. <https://www.usenix.org/conference/osdi22/presentation/yyu>
- [60] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. Hellaswag: Can a machine really finish your sentence?. In *Proceedings of the 57th annual meeting of the association for computational linguistics*. 4791–4800.
- [61] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient execution of structured language model programs. In *Proceedings of the 38th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS '24)*. Curran Associates Inc., Red Hook, NY, USA, Article 2000, 27 pages.
- [62] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, Ada Gavrilovska and Douglas B. Terry (Eds.). USENIX Association, 193–210. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>
- [63] Wenhao Zhu, Hongyi Liu, Qingxiu Dong, Jingjing Xu, Shujian Huang, Lingpeng Kong, Jiajun Chen, and Lei Li. 2024. Multilingual Machine Translation with Large Language Models: Empirical Results and Analysis. In *Findings of the Association for Computational Linguistics: NAACL 2024, Mexico City, Mexico, June 16-21, 2024*, Kevin Duh, Helena Gómez-Adorno, and Steven Bethard (Eds.). Association for Computational Linguistics, 2765–2781. doi:10.18653/V1/2024.FINDINGS-NAACL.176
- [64] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, Simon Brunner, Chen Gong, Thong Hoang, Armel Randy Zebaze, Xiaoheng Hong, Wending Li, Jean Kaddour, Ming Xu, Zhihan Zhang, Prateek Yadav, Naman Jain, Alex Gu, Zhoujun Cheng, Jiawei Liu, Qian Liu, Zijian Wang, Binyuan Hui, Niklas Muennighoff, David Lo, Daniel Fried, Xiaoning Du, Harm de Vries, and Leandro Von Werra. 2024. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. *arXiv preprint arXiv:2406.15877* (2024).